ICC-133-T Fundamentos de Sistemas Computacionales

Pontificia Universidad Católica Madre y Maestra

July 1, 2026

1 Asignación 03

1.1 Ejercicios

1. Cree una estructura en lenguaje C llamada **Estudiante** con variables miembro: nombre (char *), edad (int) y calificacion (float). Escriba un programa en C para ingresar datos de dos estudiantes, mostrar su información y encontrar el promedio de las calificaciones totales.

2. Defina una estructura denominada **Tiempo** con variables miembro: horas, minutos y segundos. Escriba un programa en C para ingresar dos horas diferentes, agregarlas y mostrar el resultado en el formato de hora adecuado.

3. Cree una estructura denominada **`Libro`** para almacenar detalles de un libro como título (char *), autor (char *) y precio (float). Escriba un programa en C para ingresar detalles de tres libros, encontrar los libros más caros y de menor precio y mostrar su información.

4. Considere las siguientes estructuras:

```
struct punto {
  int x;
  int y;
};

struct rectangulo {
  struct punto arribaIzquierda;
  struct punto abajoDerecha;
};
```

   la estructura **`punto`** representa las coordenadas x e y de un punto en un espacio bidimensional. Los puntos en la estructura **`rectángulo`** representan los puntos en las esquinas superior izquierda e inferior derecha de un rectangulo en un espacio bidimensional. Escriba y pruebe una función **`float area(struct rectangulo *rect)`** que tome una estructura rectangular como parámetro y devuelva el área del mismo.

5. Dadas las mismas estructuras que en la pregunta anterior, escriba y pruebe la función **`int hay_intercepcion(struct rectangulo *r, struct punto *p)`** que tome un rectángulo de estructura r y un punto de estructura p, y devuelva verdadero (1) si p se encuentra dentro de r y falso (0) en caso contrario.

6. Utilizando las estructuras del ejercicio 4, escriba y pruebe la función **`struct rectangulo *crear_rectangulo(int x1, int y1, int x2, int y2)`** en lenguaje C que reciba como parámetros las coordenadas del rectángulo y la función devuelva un puntero a un rectángulo creado en la memoria heap.

7. Escriba y pruebe la estructura **`complejo`** que permita almacenar la parte real y la parte imaginária de un número complejo en coordenadas rectangulares (ambos de tipo float).

8. Utilizando la estructura anterior, escriba y pruebe la función **`struct complejo *sumar_complejos(struct complejo c1, struct complejo c2)`** en lenguaje C que devuelva una estructura compleja conteniendo la suma de los dos números complejos pasados como parámetros.

9. Utilizando la estructura anterior, escriba y pruebe la función **`char *complejo_str(struct complejo c1)`** en lenguaje C que reciba como parámetro una estructura compleja y la función devuelva una cadena de caracteres representando los números complejos en formato $\mathbf{a + jb}$. Ver ejemplo:

```
struct complejo c1 = {-4.37, 9.5};
char *str = complejo_str(c1);
printf("[%s]\n", str);

// Debe de imprimir [-4.37 + j9.50]
```

10. Escriba un programa en C para crear una lista enlazada (linked list) de números enteros de n nodos y cree la función printReversed que imprima la lista en orden inverso.

```
Introduzca el número de nodos:  3
Datos de entrada para el nodo 1:  5
Datos de entrada para el nodo 2:  6
Datos de entrada para el nodo 3:  7

Los datos ingresados en la lista son:
Datos = 5
Datos = 6
Datos = 7
La lista al revés es:
Datos = 7
Datos = 6
Datos = 5
]
```

NOTA: Observe que en los problemas 6, 8 y 9 debe de utilizar el manejo dinámico de memoria con las funciones **`malloc()`** y **`free()`** de la librería **`stdlib.h`**